

DOKUMENTATION TECHNIK

Bewerbungsverwaltung · Dokumentation

Quelle: DOKUMENTATION_TECHNIK.md

Generiert am: 16.06.2026 18:58

Inhaltsverzeichnis

- Projektüberblick
- Repository-Struktur
- Datenquelle und Blätter
- Fachlogik
 - Neue Bewerbungen
 - Nachzufassen
 - Archiv
- GUI-Besonderheiten
- PDF-Dokumentationspipeline
- Workbook-Health-Check
- Entwicklung lokal
- Build (EXE)
- CI
- Bekannte technische Hinweise
- Erweiterungsvorschläge

Hinweis: Alle Einträge im Inhaltsverzeichnis sind im PDF direkt anklickbar.

Projektüberblick

Dieses Projekt stellt eine Desktop-GUI für eine bestehende Excel-basierte Bewerbungsverwaltung bereit.

- Sprache: Python 3.13
- GUI: tkinter/ttk
- Excel-Zugriff: openpyxl
- EXE-Build: PyInstaller

Repository-Struktur

- src/run.py – Einstiegspunkt
- src/bewerbungsverwaltung_app/main.py – App-Start
- src/bewerbungsverwaltung_app/gui.py – GUI (Tabs, Events, Bedienlogik)
- src/bewerbungsverwaltung_app/excel_repository.py – Excel-Lese-/Schreiblogik
- src/bewerbungsverwaltung_app/constants.py – Feld-/Spaltenzuordnung, Kategorien
- src/bewerbungsverwaltung_app/models.py – Datenmodelle
- src/bewerbungsverwaltung_app/utils.py – Konvertierung (String/Datum)
- src/build_exe.ps1 – Build-Helfer für EXE
- src/scripts/convert_docs_to_pdf.py – Markdown-zu-PDF-Konvertierung mit festem Dokumentlayout
- src/scripts/convert_docs_to_pdf.ps1 – PowerShell-Wrapper für die PDF-Erzeugung
- .github/workflows/ci.yml – CI Smoke-Checks

Datenquelle und Blätter

Primäre Datei:

- data/Bewerbungsaktivitäten mit Erinnerungen.xlsx

Relevante Blätter:

- Bewerbungsübersicht (Master-Daten)
- Hilfstabellen (Lookup-/Dropdown-Werte)
- Heute erledigen (formelbasierte Sicht)
- Diese Woche erledigen (formelbasierte Sicht)

Die Summary-Blätter werden als abgeleitete Sichten technisch repariert/neu aufgebaut, falls Formeln fehlen oder beschädigt sind.

Fachlogik

Neue Bewerbungen

Schreibt neue Zeilen in Bewerbungsübersicht.

Setzt/übernimmt Formeln für:

- Bewerbungs-ID (Spalte A)

- Erinnerungsdatum (Spalte R)
- Heute erledigen? (Spalte V)

Zusätzlich:

- Datumsfelder werden mit dd.mm.yyyy formatiert.
- Bewerbungs-ID erhält bei fehlendem Formel-Cache eine Fallback-Bildung in der Lade-Logik.

Nachzufassen

Kategorisierung auf Basis von:

- Erinnerungsdatum fällig
- Follow-up-Status älter als 14 Tage
- Feld „Heute erledigen?“ = ja

Erinnerungsdatum-Fallback:

- Wenn `erinnerungsdatum` aus Excel (z. B. `data_only=True`) leer ist, wird für Follow-up-Status bei vorhandenem `status_datum` intern `status_datum + 14 Tage` verwendet.

Archiv

Archiviert (in GUI-Sicht), wenn:

- Endergebnis gesetzt ist oder
- Status in finaler Menge (Absage, Zusage) liegt

GUI-Besonderheiten

- ttk.Notebook mit clam-Theme für reproduzierbares Tab-Styling unter Windows.
- Zusätzlicher Tab Laufende Bewerbungen mit eigener Filter-/Sortier-/Update-Logik.
- Follow-up- und Archiv-Listen mit horizontalem + vertikalem Scrollbar-Setup.
- Filter mit Debounce.
- Sortierung pro Spalte.
- Farbliche Überfälligkeitsstufen in Follow-up-Listenzeilen.
- Legende als Statusleiste im Follow-up-Tab.
- Doppelklick auf Einträge in Laufend/Nachzufassen/Archiv öffnet einen Detail-Editor (Toplevel) mit allen Feldern aus Bewerbungsübersicht.
- Detail-Editor mit Dirty-Tracking inkl. visueller Anzeige (* im Fenstertitel, Statuslabel im Fenster, kurzer Hinweis-Blink beim ersten Wechsel auf „ungespeichert“).
- Schließen-Logik im Detail-Editor über `askyesnocancel` (Speichern / Verwerfen / Abbrechen) inklusive `WM_DELETE_WINDOW`-Handling.
- Katalogverwaltung für Nächster Schritt (GUI-Dialog, Speicherung in Excel).
- Header-Schaltflächen für Schnellstart, Anwender-Doku und Technik-Doku öffnen direkt die erzeugten PDFs aus `docs/pdf/`.

PDF-Dokumentationspipeline

Die Projektdokumentation wird zusätzlich als PDF ausgeliefert. Die Generierung folgt festen Layout-Regeln:

- Seite 1 enthält ausschließlich das Titelblatt.
- Seite 2 enthält ein klickbares Inhaltsverzeichnis mit internen PDF-Links.
- Ab Seite 3 folgt der eigentliche Dokumentinhalt.
- Markdown-Titel und Markdown-Inhaltsverzeichnis werden beim PDF-Rendering aus dem Fließteil entfernt, damit keine Dubletten entstehen.
- Abschnittsüberschriften (H1–H3) werden beim Satz nach Möglichkeit mit dem unmittelbar folgenden Block zusammengehalten.

Erzeugung lokal:

- `./src/scripts/convert_docs_to_pdf.ps1`

Release-Integration:

- `src/release.ps1` erzeugt die PDFs automatisch vor dem Packen des Release-ZIP.
- Die PDFs werden zusätzlich unter `docs/pdf/` in das Release-Archiv übernommen.

Workbook-Health-Check

Beim Start führt `ExcelRepository.ensure_workbook_health()` Prüfungen/Reparaturen aus, insbesondere:

- Summary-Formeln (Heute erledigen, Diese Woche erledigen)
- Datenvalidierungen in Bewerbungsübersicht
- bedingte Formatierungen in Bewerbungsübersicht
- Seeden des Katalogs Nächster Schritt in Hilfstabellen (nur wenn leer)

Entwicklung lokal

Voraussetzungen:

- Python 3.13
- virtuelle Umgebung `.venv`

Abhängigkeiten installieren:

- `pip install -r src/requirements.txt`

Starten:

- `python src/run.py`

Hinweis: Für reproduzierbare Ergebnisse sollte die Excel-Datei während Schreibtests nicht parallel in Excel geöffnet sein.

Build (EXE)

Skript:

- ./src/build_exe.ps1

Optional clean:

- ./src/build_exe.ps1 -Clean

Ausgabe:

- dist/Bewerbungsverwaltung.exe

CI

Workflow:

- .github/workflows/ci.yml

Checks:

- Dependency-Installation
- Syntaxprüfung (py_compile)
- Excel-Smoke-Test gegen die mitgelieferte Arbeitsmappe

Bekannte technische Hinweise

- openpyxl kann Warnungen zu Data Validation Extensions ausgeben.
- Bei OneDrive kann es temporär Dateisperren geben (insb. während Sync).
- Bei EXE-Ausführung ist entscheidend, dass Excel-Datei und EXE im selben Verzeichnis liegen.
- data_only=True liefert bei ungecachten Formeln ggf. leere Werte; relevante Felder besitzen deshalb Fallback-Logik.

Erweiterungsvorschläge

- Feldvalidierung weiter ausbauen (Domänenregeln je Status)
- Such-/Filterfunktionen im Archiv
- Optionaler Wechsel auf Excel-Automation (z. B. xlwings) für komplexere Excel-Features
- Logging/Diagnosemodus für Supportfälle